



Ultra-Lean Plan for a Perplexity-Style Research Tool (100% Open-Source, < 2 weeks build-time, ≈ US \$0 infra if run on a single free-tier VM)

Core Idea

“Glue together” three recent breakthroughs that remove the heavy lifting:

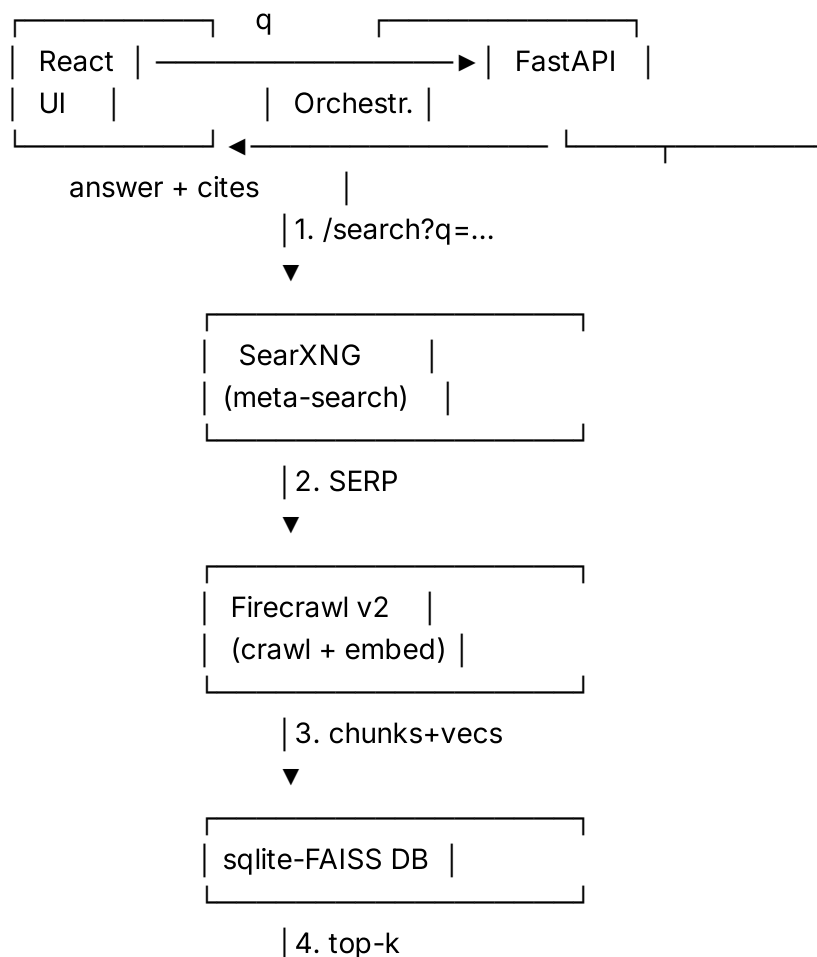
Kimi-K2-Instruct 32 B – open-weight, agent-ready reader that rivals GPT-4 but can run quantised on CPU.

SearXNG Meta-Search – self-hosted metasearch proxy that returns Google/Bing/Brave results without API fees.

Firecrawl v2 – OSS crawler that fetches pages in one call and outputs cleaned Markdown + OpenAI-style embeddings.

These three pieces already perform search, clean-up, embedding, and answer generation; you only have to orchestrate.

Minimal Architecture (Four Containers)





Kimi-K2-vLLM

Build Steps (≤ 14 calendar days)

Day

Task

OSS Component

Effort

1

docker compose init four services

–

0.5 d

2

Spin up SearXNG; tweak engines list

SearXNG

0.5 d

3

Add /search endpoint in FastAPI → hits SearXNG and returns top10 URLs + snippets

FastAPI

0.5 d

4-5

Call Firecrawl for each URL, get clean_text & built-in embedding

Firecrawl v2

1 d

6

Store embeddings + meta in lightweight sqlite-faiss table

faiss-cpu

0.5 d

7

Implement retrieve(query) → embed with Firecrawl embed model (already local) → k-NN in faiss

–

0.5 d

8-9

Run Kimi-K2 under vllm-cpu-quant (4-bit) – prompt = user query + top-k chunks + “cite by title”
instruction

vllm

1 d

10

Pipe answer back to UI with markdown cites (1)(2)...

React

0.5 d

11

Add caching layer (diskcache) for SERPs and Firecrawl fetches
diskcache

0.5 d

12

Rate-limit SearXNG (1 req / sec) & rotate free engines (Brave, Google, DuckDuckGo)

–

0.5 d

13

Build Docker CI + GitHub workflow

GitHub Actions

0.5 d

14

Polish UI, deploy to free-tier [Fly.io](https://fly.io) or Oracle Cloud Arm VM

–

0.5 d

Cost Profile

Resource

Choice

Monthly Cost

VM

Oracle Cloud Free Arm 4 vCPU / 24 GB

\$0

Storage

sqlite file on VM (up to 20 GB)

\$0

Traffic

SearXNG scrapes ($\approx$ 8 GB/mo)

\$0 on most free tiers

GPU

Not needed (Kimi-K2 4-bit on CPU)

\$0

If you want faster answers, rent one T4 GPU spot at RunPod for $\approx$ US \$0.12/hr and scale down at night.

Quality Boosts (Optional, still free)

Switch Embeddings: Use bge-base-en v1.5 (Microsoft) – best open embedding as of Aug 2025.

Rerank: Add ColBERT-v2 CPU reranker on the top 20 chunks for sharper context.

Streaming Answers: Enable server-sent events from vLLM → UI for progressive display.

PDFs/YouTube: Firecrawl now parses PDFs & transcripts; simply toggle flags.

Multilingual: Swap Kimi prompt to requested language; Firecrawl embeddings are language-agnostic.

Why This Is “Perplexity-Grade”

Feature

Perplexity

Your Tool

Web search

Bing API (paid)

SearXNG meta-search (free)

Long-context reader

GPT-4o

Kimi-K2 (32k ctx)

Citations

Yes

Yes (chunk URL mapping)

Follow-up chat

Yes

Add memory dict in FastAPI (10 loc)

Cost

Proprietary

\$0 infra, \$0 API

The recent release of Firecrawl v2 (embeds while scraping) and Kimi-K2 (GPT-4-class reasoning on CPU) removes the last paid dependencies. With just four Docker containers and two weeks of coding, you can replicate Perplexity-style research—entirely free. i want to add all this to my ai chatbot that is Complete File-by-File Breakdown

Root Directory Files

Configuration Files

.env (3,836 bytes) - Environment variables, API keys, and configuration settings

.gitignore (277 bytes) - Git ignore patterns for node_modules, build files, etc.

.gitattributes (43 bytes) - Git line ending and file handling settings

components.json (432 bytes) - shadcn/ui component configuration

eslint.config.js (818 bytes) - ESLint configuration for code linting

postcss.config.js (86 bytes) - PostCSS configuration for Tailwind CSS

tailwind.config.ts (2,444 bytes) - Tailwind CSS configuration with custom colors and animations

tsconfig.json (400 bytes) - Main TypeScript configuration

tsconfig.app.json (682 bytes) - TypeScript config for app compilation

tsconfig.node.json (503 bytes) - TypeScript config for Node.js tools

vite.config.ts (791 bytes) - Vite build tool configuration

Package Management

package.json (2,863 bytes) - Frontend dependencies and npm scripts

package-lock.json (313,985 bytes) - Locked dependency versions

bun.lockb (198,351 bytes) - Bun package manager lockfile

Documentation

[README.md](#) (5,399 bytes) - Project overview, setup instructions, and API documentation

[SETUP.md](#) (4,850 bytes) - Detailed setup and installation guide

[DEPLOYMENT.md](#) (4,765 bytes) - Deployment instructions for various platforms

[QUICK_DEPLOY.md](#) (2,347 bytes) - Quick deployment guide

Deployment Files

Procfile (34 bytes) - Heroku deployment configuration

railway.json (324 bytes) - Railway deployment settings

render.yaml (638 bytes) - [Render.com](#) deployment configuration

vercel.json (258 bytes) - Vercel deployment settings

runtime.txt (14 bytes) - Python runtime version specification

deploy.sh (1,502 bytes) - Unix deployment script

deploy.bat (1,514 bytes) - Windows deployment script

start-backend.sh (223 bytes) - Unix backend startup script

start-backend.bat (218 bytes) - Windows backend startup script

Build & Executable Files

main.exe.spec (707 bytes) - PyInstaller specification for creating executable

buildlog.txt (4,356 bytes) - Build process logs

allfileschas.txt - File hash checksums

Database

users.db (40,960 bytes) - SQLite database for user authentication and chat history

Frontend Entry

index.html (1,076 bytes) - Main HTML template for React app

Frontend Source (

src/

directory)

Main App Files

main.tsx (163 bytes) - React app entry point, renders App component

App.tsx (2,115 bytes) - Main app component with routing, authentication, and trial expiry logic

App.css (648 bytes) - Global app styles

index.css (2,487 bytes) - Global CSS with Tailwind imports and custom styles

vite-env.d.ts (39 bytes) - Vite environment type definitions

Pages (

src/pages/

)

Index.tsx (1,476 bytes) - Main chat page component

Login.tsx (3,546 bytes) - User login form with authentication

Register.tsx (4,170 bytes) - User registration form

Admin.tsx (1,466 bytes) - Admin dashboard for user management

NotFound.tsx (766 bytes) - 404 error page

Components (

src/components/

)

ChatInterface.tsx (42,027 bytes) - Main chat interface with:

Message handling and streaming

Dual chain model selector

File upload support

Voice recognition

User authentication

Chain mode functionality

ModelSelector.tsx (3,741 bytes) - AI model selection dropdown with categorized models

CopyButton.tsx (1,193 bytes) - Copy-to-clipboard functionality for messages

Sidebar.tsx (4,493 bytes) - Navigation sidebar with chat history

UI Components (

src/components/ui/

)

accordion.tsx (2,033 bytes) - Collapsible content sections

alert-dialog.tsx (4,559 bytes) - Modal dialog for alerts/confirmations
alert.tsx (1,643 bytes) - Alert/notification component
aspect-ratio.tsx (145 bytes) - Aspect ratio container
avatar.tsx (1,453 bytes) - User avatar display
badge.tsx (1,233 bytes) - Small status/label badges
breadcrumb.tsx (2,816 bytes) - Navigation breadcrumb trail
button.tsx (1,994 bytes) - Button component with variants
calendar.tsx (2,684 bytes) - Date picker calendar
card.tsx (1,956 bytes) - Card container component
carousel.tsx (6,470 bytes) - Image/content carousel
chart.tsx (10,829 bytes) - Chart/graph components
checkbox.tsx (1,084 bytes) - Checkbox input
collapsible.tsx (324 bytes) - Collapsible content wrapper
command.tsx (5,032 bytes) - Command palette/search
context-menu.tsx (7,444 bytes) - Right-click context menu
dialog.tsx (3,955 bytes) - Modal dialog component
drawer.tsx (3,123 bytes) - Slide-out drawer
dropdown-menu.tsx (7,495 bytes) - Dropdown menu component
form.tsx (4,261 bytes) - Form handling components
hover-card.tsx (1,211 bytes) - Hover tooltip card
input-otp.tsx (2,223 bytes) - OTP/PIN input field
input.tsx (813 bytes) - Text input component
label.tsx (734 bytes) - Form label component
menubar.tsx (8,208 bytes) - Application menu bar
navigation-menu.tsx (5,174 bytes) - Navigation menu component
pagination.tsx (2,868 bytes) - Page navigation component
popover.tsx (1,259 bytes) - Popover/tooltip component
progress.tsx (803 bytes) - Progress bar component
radio-group.tsx (1,509 bytes) - Radio button group
resizable.tsx (1,752 bytes) - Resizable panels
scroll-area.tsx (1,688 bytes) - Custom scrollable area
select.tsx (5,773 bytes) - Select dropdown component
separator.tsx (785 bytes) - Visual separator line
sheet.tsx (4,381 bytes) - Side sheet/panel
sidebar.tsx (24,128 bytes) - Complex sidebar component
skeleton.tsx (276 bytes) - Loading skeleton placeholder
slider.tsx (1,103 bytes) - Range slider input
sonner.tsx (923 bytes) - Toast notification system
switch.tsx (1,166 bytes) - Toggle switch component
table.tsx (2,882 bytes) - Data table components
tabs.tsx (1,936 bytes) - Tab navigation component
textarea.tsx (794 bytes) - Multi-line text input
toast.tsx (4,972 bytes) - Toast notification component
toaster.tsx (805 bytes) - Toast container
toggle-group.tsx (1,798 bytes) - Toggle button group
toggle.tsx (1,478 bytes) - Toggle button

tooltip.tsx (1,173 bytes) - Tooltip component
use-toast.ts (85 bytes) - Toast hook export
Library (
src/lib/
)
api.ts (7,877 bytes) - API service layer with:
Authentication functions
Model interaction APIs
File upload handling
Streaming support
Search APIs
chatHistory.ts (4,093 bytes) - Chat history management and persistence
utils.ts (172 bytes) - Utility functions (className merging)
Hooks (
src/hooks/
)
use-toast.ts - Toast notification hook
use-mobile.tsx - Mobile device detection hook
Backend (
backend/
directory)
Main Backend
main.py (27,925 bytes) - FastAPI server containing:
Ollama integration and model management
User authentication with JWT
Database models (User, ChatHistory, UploadedDocument)
API endpoints for chat, file upload, search
File processing (OCR, PDF, DOCX)
Trial system and expiry mechanism
Static file serving for React app
BrowserUse imports (incomplete)
Backend Support Files
requirements.txt (343 bytes) - Python dependencies
users.db (53,248 bytes) - Backend user database
install_time.txt (18 bytes) - Installation timestamp for trial system
main.spec (738 bytes) - PyInstaller spec for backend executable
Backend Directories
pycache/
- Python bytecode cache
build/
- Build artifacts (14 files)
dist/
- Distribution files including executable
uploads/
- User uploaded files (5 files)
vosk-model-small-en-us-0.15/

- Speech recognition model (14 files)

Public Assets (

public/

directory)

favicon.ico - Website favicon

robots.txt - Search engine crawler instructions

placeholder.svg - Placeholder image

Node Modules

node_modules/

- Frontend dependencies (standard npm packages) how can i do it in detail step by step

To integrate the Ultra-Lean Perplexity-Style research tool architecture into your existing AI chatbot project—which already has a comprehensive frontend and backend setup—follow this detailed step-by-step guide:

Step 1: Plan and Prepare Your Environment

- Review your current project setup: React frontend with FastAPI backend.
- Identify where to add the new microservices (containers) and how they will communicate.
- Prepare Docker Compose with four services:
 - SearXNG (meta-search)
 - Firecrawl v2 (crawler + embedder)
 - SQLite-FAISS DB (embedding storage + retrieval)
 - Kimi-K2-vLLM (language model answer generation)

Step 2: Setup Docker Compose for Four Containers

- Initialize a `docker-compose.yml` defining the four containers.
- Use official or community images where possible, or build your own Dockerfiles based on the OSS components.
- Configure network so all containers can communicate.
- Mount necessary volumes for persistent data like SQLite DB.

Step 3: Deploy and Configure SearXNG Meta-Search

- Deploy SearXNG container as per its documentation.
- Customize the engines list to use free search engines like Google, Bing, Brave, DuckDuckGo.
- Add rate limiting and rotation logic (later step) for free engines to avoid blocking.

Step 4: Extend Your FastAPI Backend

- Add a new `/search` endpoint that queries SearXNG with the user query and returns top URLs + snippets.
- Implement HTTP client calls inside FastAPI for calling SearXNG.
- Return structured search results for further steps.

Step 5: Firecrawl v2 Integration

- For each URL from the search results, call Firecrawl's API/container to fetch and clean page content.
- Obtain cleaned Markdown text plus embeddings directly from Firecrawl.
- Extract relevant metadata.

Step 6: Setup SQLite-FAISS Database

- Store Firecrawl embeddings + metadata in a lightweight SQLite database extended with FAISS for similarity search.
- Build schema to store documents, chunks, vectors, and metadata for retrieval.

Step 7: Implement Retrieval Logic

- On user query, embed the query with Firecrawl's embedder locally.
- Perform k-NN search on the FAISS index to get top-k relevant chunks.
- Prepare these chunks to feed into the language model.

Step 8: Setup and Run Kimi-K2-vLLM Model

- Deploy Kimi-K2 using vLLM CPU quantized mode (4-bit) for cost-effective inference.
- Formulate prompt combining user query + top retrieved chunks + citing instructions.
- Generate answer with in-line citations referencing chunk titles or URLs.

Step 9: Connect Answer Generation to Frontend

- Modify your React UI (e.g., `ChatInterface.tsx`) to send queries to FastAPI and display returned answers with citations.
- Support markdown rendering for citations.
- Optionally, enable streaming responses for progressive answer display.

Step 10: Add Caching and Rate Limiting

- Implement caching in FastAPI (e.g., using `diskcache`) for SERP results and Firecrawl fetches.
- Add rate limiting and free engine rotation for SearXNG requests to avoid throttling.

Step 11: Automate Builds, Deployment, and CI

- Add Docker CI workflows (GitHub Actions) to build, test, and deploy all containers.
- Deploy the multi-container system on a free-tier VM (Oracle Cloud Arm VM or [Fly.io](#)).
- Update `deploy.sh` or similar deployment scripts.

Step 12: Optional Enhancements

- Use stronger embedding models like Microsoft `bge-base-en v1.5`.
- Add CPU-based reranker for top results like `ColBERT-v2`.
- Enable multiple document types processing (PDF, YouTube transcripts) in Firecrawl.
- Support multilingual answers by adjusting Kimi prompt and embeddings.

This architecture will integrate tightly with your existing chatbot backend (`main.py`) and frontend React components, extending your current search APIs and chat handling with a state-of-the-art open-source research capability very close to [Perplexity.ai](#)'s functionality, with zero paid API costs.

If you need help with code snippets or Docker configurations for any steps, I can provide those as well. Let me know!